

**Yenepoya Institute of Arts, Science,
Commerce, and Management
Final Project Report**

On

Personalised Recommendation Engine for Retail

Student Name: Niha Najeeb

Industry Mentor: Mr. Sumit

Guided by: Mrs. Ankitha

TABLE OF CONTENTS

1. Executive Summary	1
2. Background	3
2.1 Aim	5
2.2 Technologies	6
2.3 Hardware Architecture	8
2.4 Software Architecture	10
3. System	13
3.1 Requirements	14
3.1.1 Functional Requirements	14
3.1.2 User Requirements	15
3.1.3 Environmental Requirements	16
3.2 Design and Architecture	17
3.3 Implementation	20
3.4 Testing	26
3.5 Graphical User Interface (GUI) Layout	29
3.6 Customer Testing	32
3.7 Evaluation	34
4. Snapshots of the Project	37
5. Conclusion	40
6. Further Development / Research	42
7. References	45
8. Appendix	47
8.1 Weekly Progress Reports (WPPR)	48

1. Executive Summary

This project presents the design and development of a content-based Personalized Recommendation Engine for Retail. The system suggests similar products to users based on descriptive product attributes such as category, usage, and features. It was developed as part of the BSc (Artificial Intelligence, Machine learning, and Data science) programme at Yenepoya Institute, in collaboration with NCS and IBM Innovation Centre for Education.

The recommendation engine uses CountVectorizer for feature extraction and Cosine Similarity for computing product similarity. Product attributes (tags) are preprocessed, vectorized into a sparse matrix, and a similarity matrix is computed once at initialization. When a user selects a product, the engine retrieves the top-N most similar products and presents them through an interactive dashboard built with ipywidgets and HTML product cards.

The system follows a layered modular architecture: a Data Layer, Processing Layer, Feature Extraction Layer, Logic Layer, and Presentation Layer. Each module is independently testable and the design supports future scalability to larger datasets and cloud deployment.

Testing was conducted through manual functional and data consistency scenarios. The dashboard was evaluated by peers and received positive feedback on usability and relevance of recommendations. The project successfully demonstrates an end-to-end machine learning pipeline from raw product data to interactive recommendations.

2. Background

The exponential growth of e-commerce has created a critical need for intelligent product discovery systems. Traditional browsing requires users to manually explore catalogues, which reduces conversion rates and user satisfaction. Recommendation engines address this by automatically surfacing relevant products, increasing engagement and sales. This project builds a practical, academic-grade recommendation system that demonstrates the full pipeline from raw data to interactive output.

The project was initiated as a data-science and ML exercise to understand content-based filtering, a foundational technique in recommender systems. Unlike collaborative filtering (which requires user interaction history), content-based filtering relies solely on product attributes, making it suitable for cold-start scenarios and smaller retail datasets.

2.1 Aim

The primary aim is to design and implement a personalized product recommendation engine for retail using content-based filtering. Specific goals include:

- Building a preprocessing and vectorization pipeline for product tag data.
- Computing a cosine similarity matrix for all product pairs.
- Implementing a recommendation function that returns the top-N similar products for any given item.
- Rendering results in an interactive shopping dashboard using ipywidgets and HTML cards.
- Producing clean, modular code following a layered architecture suitable for academic evaluation and future extension.

2.2 Technologies

The project uses lightweight Python data-science tools appropriate for rapid ML prototyping:

- Python 3.x – core programming language.
- Pandas – dataset management and DataFrame operations.
- Scikit-learn (CountVectorizer, cosine_similarity) – feature extraction and similarity computation.
- ipywidgets – interactive dropdown and button widgets for the dashboard.
- IPython.display / HTML – product card rendering inside Jupyter Notebook.

- Jupyter Notebook – development and demonstration environment.
- NumPy – numerical operations supporting the similarity matrix.

2.3 Hardware Architecture

The system runs on a single-machine development environment suitable for academic demonstration:

- Client Side: Jupyter Notebook rendered in a browser (Chrome/Edge/Firefox).
- Compute: Local machine executes Python, pandas, and scikit-learn operations in memory.
- Storage: Product dataset stored as an in-memory Pandas DataFrame (CSV-importable).

Minimum recommended hardware:

- Processor: Dual-core or above
- RAM: 4 GB minimum (8 GB recommended for larger datasets)
- Storage: 1 GB free disk space
- Network: Not required for local execution

This architecture is intentionally compact to enable clear end-to-end demonstration without distributed infrastructure complexity.

2.4 Software Architecture

The software follows a five-layer modular design:

1. Data Layer – Stores product dataset (item, tags, price, rating) in a Pandas DataFrame.
2. Processing Layer – Handles text normalization: lowercase conversion, tokenization, and removal of inconsistencies.
3. Feature Extraction Layer – Converts processed tags into numerical vectors using CountVectorizer (Bag-of-Words).
4. Logic Layer – Computes an NxN cosine similarity matrix; implements ranking and top-N retrieval logic.
5. Presentation Layer – Renders interactive dropdown and product cards using ipywidgets and HTML templates.

Architectural characteristics:

- Stateless recommendation calls (each query is processed independently).
- In-memory similarity matrix for fast retrieval.
- Modular helper functions for each processing stage.
- Loose coupling between layers enables independent testing and future replacement.

3. System Design

3.1 Application Design

At the implementation level, the application consists of tightly integrated modules:

- Data Initialization Module: Loads dataset into a Pandas DataFrame and ensures correct schema.
- Preprocessing Module: Handles text normalization (lowercase, tokenization, removal of inconsistencies).
- Feature Extraction Module: Converts processed text into numerical vectors using CountVectorizer.
- Similarity Computation Module: Generates an NxN similarity matrix using cosine similarity.
- Recommendation Engine: Applies ranking logic, excludes self-comparison, applies fallback logic, and retrieves top-N similar products.
- UI Rendering Module: Uses ipywidgets and HTML templates to display product cards dynamically.

3.2 Process Flow

Detailed execution pipeline:

6. System initializes and loads dataset.
7. Tags are normalized (lowercase, cleaned).
8. Vocabulary is created using CountVectorizer.
9. Tags are transformed into a sparse matrix.
10. Cosine similarity matrix is computed (NxN matrix).
11. User selects a product from the dropdown widget.
12. Product index is retrieved using DataFrame lookup.
13. Corresponding similarity scores are fetched from the matrix.
14. Scores are sorted in descending order.
15. Top-N products (excluding itself) are selected.
16. Results are formatted for display.
17. UI renders product cards with price, rating, and similarity score.

3.3 Information Flow

Internal data flow:

- User Input: Dropdown selection triggers recommendation function.
- Input passed to recommendation function with the selected product name.

- Function queries the DataFrame for the product index.
- Index used to retrieve the corresponding row of similarity scores from the matrix.
- Scores are processed, sorted, and top-N selected.
- Result data structured into a display format (list of product dicts).
- Output passed to the UI renderer.
- UI displays product cards dynamically with price, rating, and similarity percentage.

3.4 Components Design

1. Dataset Manager

- Stores product data in a Pandas DataFrame.
- Provides access methods (filter, index-based retrieval, lookup by name).
- Ensures data consistency and correct schema on load.

2. Preprocessing Module

- Converts text to lowercase.
- Removes unwanted characters and excess whitespace.
- Standardizes tag format for consistent vectorization.
- Handles missing values (fills or drops as appropriate).

3. Vectorization Module

- Uses CountVectorizer from scikit-learn.
- Converts text into Bag-of-Words (BoW) representation.
- Generates a sparse feature matrix of shape (n_products, n_vocabulary).

4. Similarity Engine

- Computes pairwise cosine similarity between all product vectors.
- Produces a dense similarity matrix of shape (NxN).
- Stores the matrix in memory for O(1) lookup per query.

5. Recommendation Module

- Retrieves similarity scores for the selected product.
- Sorts products by descending similarity score.
- Excludes the query product (self-comparison).
- Applies fallback logic when similarity scores are all zero.
- Returns the top-N recommendations as a list.

6. UI Module

- Implements dropdown and button using ipywidgets.
- Displays output using HTML templates inside Jupyter output cells.
- Generates product cards dynamically with price, rating, and match score.
- Handles user interaction events (on_click callback).

3.5 Key Design Considerations

- **Efficiency:** Vectorized operations (NumPy/scikit-learn) for fast batch computation.
- **Scalability:** Modular design supports dataset expansion and future database integration.
- **Reusability:** Each component can be reused across different retail domains.
- **User Experience:** Interactive dropdown UI improves discoverability and usability.
- **Maintainability:** Clean separation of preprocessing, logic, and presentation layers.

3.6 API Catalogue

API Name	Description	Input	Output
load_data()	Loads dataset into memory	File path	DataFrame
preprocess_data()	Cleans and prepares tags	Raw data	Processed data
vectorize_data()	Converts tags to vectors	Tags	Feature matrix
compute_similarity()	Generates similarity matrix	Vectors	NxN Matrix
get_index()	Fetch product index	Product name	Index (int)
recommend_product()	Returns top-N recommendations	Product name	List of items
render_cards()	Displays product cards in UI	Product list	HTML output

4. Graphical User Interface Layout

The GUI is implemented inside a Jupyter Notebook as an interactive dashboard with the following layout:

- Header Section: Project title and brief description of the recommendation engine.
- Product Selection: Dropdown widget listing all available products from the dataset.
- Action Control: 'Get Recommendations' button that triggers the recommendation function.
- Results Section: Dynamically rendered HTML product cards for each recommendation, each showing:
 - Product name and tags.
 - Price (in INR) and customer rating.
 - Cosine similarity percentage with the selected product.

UI Design Principles Followed:

- Consistent card-based layout for easy visual scanning.
- Immediate feedback rendered below the action button.
- Readable typography with clear hierarchy (product name > price/rating > similarity).
- Minimal interaction friction: single dropdown + single button to get results.

5. Conclusion

This project demonstrates the successful design and implementation of a content-based personalized recommendation engine for retail. The system validates core ML pipeline concepts by integrating data preprocessing, feature extraction, similarity computation, and an interactive UI into one cohesive application.

The implemented architecture supports practical recommendation workflows and provides a solid academic foundation for understanding recommender system design. The modular approach ensures each component is independently understandable, testable, and extendable. From an academic perspective, the project satisfies objectives related to machine learning pipeline design, implementation, testing, interface development, and system evaluation. It also provides a strong base for future production-oriented enhancements.

6. Further Development or Research

Recommended future scope:

- Integrate TF-IDF vectorization to improve feature weighting over BoW.
- Add word embeddings (Word2Vec, GloVe, BERT) for semantic similarity.
- Implement collaborative filtering to complement content-based recommendations.
- Build a hybrid recommender combining content-based and collaborative signals.
- Deploy as a REST API using Flask or FastAPI for web/mobile consumption.
- Add user authentication and personalized recommendation history.
- Integrate a relational database (PostgreSQL) for persistent product storage.
- Implement evaluation metrics (Precision@K, Recall@K, NDCG) for quality measurement.
- Deploy to cloud (AWS/GCP) with CI/CD pipeline for continuous updates.
- Add an admin dashboard for product and recommendation management.
- Apply approximate nearest neighbour search (FAISS, Annoy) for large-scale efficiency.

Research directions:

- Multilingual product tag handling for regional retail markets.
- Real-time similarity update as new products are added (incremental vectorization).
- Graph-based recommendation using product co-purchase networks.
- Explainable AI techniques to show users why a product was recommended.

7. References

- Python Software Foundation. Python Documentation. <https://docs.python.org/3/>
- Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR, 2011.
- McKinney, W. Data Analysis with Pandas. O'Reilly Media.
- Ricci, F., Rokach, L., Shapira, B. Recommender Systems Handbook. Springer.
- Manning, C.D., Raghavan, P., Schutze, H. Introduction to Information Retrieval. Cambridge University Press.
- Jupyter Project Documentation. <https://jupyter.org/documentation>
- Scikit-learn Documentation. <https://scikit-learn.org/stable/>
- NumPy Documentation. <https://numpy.org/doc/>

8. APPENDIX

The Retail Recommendation Dashboard is implemented using Python and follows a content based filtering approach. The system utilizes libraries such as Pandas for data handling and Scikit-learn for machine learning operations. The core logic involves converting product tags into numerical vectors using Count Vectorizer and computing similarity between products using cosine similarity. Based on this, the system recommends similar items when a user selects a product.

For example, when a user selects a product like T-Shirt, the system analyses its features and suggests related items such as Jeans, Jacket, and Sneakers. The output is displayed through an interactive dashboard built using Jupyter Notebook and ipywidgets, where recommendations are presented in a structured and user-friendly format.

The system operates on a predefined dataset and does not include real-time data or user personalization. While it performs efficiently for small to medium datasets, it has limitations in scalability and dynamic data handling. Future improvements may include adding product images, integrating real-time data sources, implementing user-based recommendations, and deploying the system as a web application for broader accessibility.

8.2.WEEKLY REPORT

Weekly Project Progress Report (WPPR-1)

For week commencing 02 March 2026 | Date: 02-03-2026

Student Name:	Niha Najeeb
Register No.	23BSAMB024
Programme:	BSc(Artificial Intelligence, Machine learning, and Data Science)
Guide:	Mrs. Ankitha Industry Mentor: Mr. Sumit
Project Title:	Personalized Recommendation Engine for Retail

Targets Set for the Week

- Set up the Jupyter Notebook development environment.
- Prepare the initial product dataset with item, tags, price, and rating fields.
- Implement data loading using Pandas.
- Explore CountVectorizer and understand Bag-of-Words representation.

Progress / Achievements

- Configured Jupyter Notebook with Python 3.x, pandas, and scikit-learn.
- Created an initial product dataset with 20 retail items across electronics, clothing, and home appliances categories.
- Implemented load_data() to read the CSV into a Pandas DataFrame with schema validation.
- Explored CountVectorizer on sample tags and verified vocabulary generation.
- Documented data model fields (item, tags, price, rating).

Future Work Plans

- Implement the preprocessing module (lowercase, tokenization, cleaning).
- Apply Count Vectorizer to the full product tag column.
- Begin cosine similarity computation.
- Structure code into modular functions for each processing stage.

Implementation Shown: ☐ Yes ☐ No

Remarks by the Internal Guide: _____

Signature of the Student: _____

Signature of the Internal Guide:

Weekly Project Progress Report (WPPR-2)

For week commencing 09 March 2026 | Date: 09-03-2026

Student Name:	Niha Najeeb
Register No.	23BSAMB024
Programme:	BSc(Artificial Intelligence, Machine learning, and Data Science)
Guide:	Mrs. Ankitha Industry Mentor: Mr. Sumit
Project Title:	Personalized Recommendation Engine for Retail

Targets Set for the Week

- Implement the preprocessing module for tag normalization.
- Apply CountVectorizer to generate the feature matrix.
- Compute the cosine similarity matrix.
- Validate matrix dimensions and diagonal values.

Progress / Achievements

- Implemented preprocess_data() to lowercase all tags, remove special characters, and handle missing values.
- Applied CountVectorizer to generate a sparse feature matrix of shape (n_products, n_vocabulary).
- Computed the NxN cosine similarity matrix using sklearn.metrics.pairwise.cosine_similarity.
- Verified matrix symmetry ($\text{sim}[i][j] == \text{sim}[j][i]$) and diagonal values equal to 1.0.
- Created helper functions get_index() and recommend_product() with top-N retrieval logic.

Future Work Plans

- Develop the UI rendering module using ipywidgets and HTML templates.
- Build the product card HTML template with price, rating, and similarity fields.
- Integrate dropdown widget with the recommendation function.
- Test end-to-end recommendation flow for several sample products.

Implementation Shown: ☐ Yes ☐ No

Remarks by the Internal Guide: _____

Signature of the Student: _____

Signature of the Internal Guide:

Weekly Project Progress Report (WPPR-3)

For week commencing 16 March 2026 | Date: 16-03-2026

Student Name:	Niha Najeeb
Register No.	23BSAMB024
Programme:	BSc(Artificial Intelligence, Machine learning, and Data Science)
Guide:	Mrs. Ankitha Industry Mentor: Mr. Sumit
Project Title:	Personalized Recommendation Engine for Retail

Targets Set for the Week

- Build the UI rendering module with ipywidgets dropdown and button.
- Design and implement HTML product card templates.
- Integrate UI with the recommendation engine function.
- Test recommendation output for multiple product selections.

Progress / Achievements

- Implemented dropdown widget populated with all product names from the DataFrame.
- Created render_cards() function that generates HTML product cards for each recommended item, displaying name, tags, price, rating, and similarity score
- Integrated the button on_click callback to call recommend_product() and render results dynamically.
- Tested recommendations for 10 different products and verified that results were relevant and self-comparison was excluded.
- Handled edge case: products with all-zero similarity scores using fallback logic returning least dissimilar items.

Future Work Plans

- Refine UI styling for product cards (colour coding by similarity score).
- Expand dataset to 50+ products across more categories.
- Conduct data consistency tests on the similarity matrix.
- Begin writing inline code documentation and module docstrings.

Implementation Shown: ☐ Yes ☐ No

Remarks by the Internal Guide: _____

Signature of the Student: _____

Signature of the Internal Guide: _____

Weekly Project Progress Report (WPPR-5)

For week commencing 23 March 2026 | Date: 23-03-2026

Student Name:	Niha Najeeb
Register No.	23BSAMB024
Programme:	BSc(Artificial Intelligence, Machine learning, and Data Science)
Guide:	Mrs. Ankitha Industry Mentor: Mr. Sumit
Project Title:	Personalized Recommendation Engine for Retail

Targets Set for the Week

- Refine product card UI styling and add similarity colour indicators.
- Expand the product dataset to 50+ items.
- Conduct comprehensive data consistency and functional testing.
- Write module docstrings and inline comments.

Progress / Achievements

- Added colour-coded similarity badges to product cards (green > 70%, amber 40-70%, red < 40%).
- Expanded the dataset to 55 products spanning electronics, clothing, home appliances, books, and sports categories.
- Ran comprehensive testing: verified matrix symmetry, correct top-N count, correct price and rating rendering on cards.
- Wrote complete docstrings for all six module functions following NumPy docstring conventions.
- Fixed a tag normalization bug where compound words were being split incorrectly, improving recommendation relevance.

Future Work Plans

- Perform peer usability testing of the dashboard.
- Collect and document feedback on recommendation quality and UI clarity.
- Begin drafting the HLD and LLD documents.
- Explore TF-IDF as an alternative to CountVectorizer.

Implementation Shown: ☐ Yes ☐ No

Remarks by the Internal Guide: _____

Signature of the Student: _____

Signature of the Internal Guide:

Weekly Project Progress Report (WPPR-5)

For week commencing 30 March 2026 | Date: 30-03-2026

Student Name:	Niha Najeeb
Register No.	23BSAMB024
Programme:	BSc(Artificial Intelligence, Machine learning, and Data Science)
Guide:	Mrs. Ankitha Industry Mentor: Mr. Sumit
Project Title:	Personalized Recommendation Engine for Retail

Targets Set for the Week

- Conduct peer usability testing and collect feedback.
- Draft High-Level Design (HLD) document.
- Draft Low-Level Design (LLD) document.
- Compare CountVectorizer vs TF-IDF performance on the dataset.

Progress / Achievements

- Conducted usability testing with 5 peers. All users successfully obtained recommendations within two interactions. Feedback was positive on card layout and similarity scores.
- Drafted the HLD document covering system architecture, component design, API catalogue, data model, and non-functional requirements.
- Drafted the LLD document with detailed module-level design, execution pipeline, and internal data flow.
- Compared CountVectorizer and TF-IDF: TF-IDF showed marginally better discrimination on common tags; CountVectorizer retained for simplicity and reproducibility.
- Documented all known limitations (NxN scaling, semantic blindness, no collaborative signals).

Future Work Plans

- Finalize code with complete documentation and clean up the notebook.
- Prepare README with setup and run instructions.
- Finalize the Final Project Report.
- Push code and documentation to the GitHub repository.

Implementation Shown: ☐ Yes ☐ No

Remarks by the Internal Guide: _____

Signature of the Student: _____

Signature of the Internal Guide:

Weekly Project Progress Report (WPPR-6)

For week commencing 13 April 2026 | Date: 13-04-2026

Student Name:	Niha Najeeb
Register No.	23BSAMB024
Programme:	BSc(Artificial Intelligence, Machine learning, and Data Science)
Guide:	Mrs. Ankitha Industry Mentor: Mr. Sumit
Project Title:	Personalized Recommendation Engine for Retail

Targets Set for the Week

- Finalize and clean up the complete Jupyter Notebook codebase.
- Prepare README.md with setup, run instructions, and dataset description.
- Finalize the Final Project Report integrating HLD and LLD content.
- Push final codebase and documentation to GitHub.
- Prepare live demonstration for final evaluation.

Progress / Achievements

- Completed final code review: refactored all six modules for readability and removed redundant cells.
- Wrote README.md documenting environment setup (pip install -r requirements.txt), dataset format, and how to run the notebook.
- Finalized the Final Project Report incorporating HLD architecture diagrams (described), LLD execution pipeline, API catalogue, data model, and testing results.
- Successfully initialized Git, committed all files with semantic commit messages, and pushed to GitHub.
- Prepared a live demonstration script covering: dataset loading, preprocessing, vectorization, similarity computation, and interactive recommendation showcase.

Future Work Plans

- Rehearse the live demonstration for the final evaluation.
- Prepare presentation slides summarizing architecture, key design decisions, and evaluation.
- Explore further development directions for the project.

Implementation Shown: ☐ Yes ☐ No

Remarks by the Internal Guide: _____

Signature of the Student: _____

Signature of the Internal Guide: _____